SPRING BOOT · API REST

Repaso API REST en capas

Editorial — Autor / Libro

Práctica previa al TP · Spring Data JPA · DTOs · Excepciones · Swagger



OBJETIVO

¿Qué vamos a construir hoy?

Una **API REST en capas** para la gestión de una editorial, aplicando el stack completo que necesitás para el TP.

Spring Data JPA

Persistencia con queries derivadas y repositorios.

DTOs

Patrón Create / Dto / Edit con mapeo propio en cada record.

Manejo de excepciones

@RestControllerAdvice centralizado, sin try/catch en cada endpoint.

Swagger / OpenAPI

Documentación automática de todos los endpoints.

⚠️ El dominio es **Autor/Libro** — distinto del TP real (Usuario/Pedido/Producto/Categoría) para que practiquen sin copiar.

MODELO DE DOMINIO

Entidades: Base · Autor · Libro

Clase abstracta Base

```
@MappedSuperclass
public abstract class Base {
  @Id @GeneratedValue
  private Long id;
  private boolean eliminado = false;
  private LocalDateTime createdAt;
}
```

Enum EstadoLibro

```
public enum EstadoLibro {
  DISPONIBLE, AGOTADO, DESCATALOGADO
}
```

Autor y Libro

```
@Entity
public class Autor extends Base {
  private String nombre, apellido, nacionalidad;
  @OneToMany(mappedBy = "autor")
  private List<Libro> libros;
}
@Entity
public class Libro extends Base {
  private String titulo;
  private Integer anio, stock;
  private Double precio;
  @Enumerated(EnumType.STRING)
  private EstadoLibro estado;
  @ManyToOne
  private Autor autor;
}
```

CONSIGNA — PASO A PASO

Hoja de ruta del ejercicio

01

Dependencia Swagger

Agregar `springdoc-openapi-starter-webmvc-ui` .

02

Repository

JpaRepository con queries derivadas (`findByEliminadoFalse`, findByAutor_IdAndEliminadoFalse).

03

DTOs

Un record Create, un record Dto, un record Edit por entidad. Cada uno lleva su propio método de mapeo.

04

Service

Interfaz + implementación. CRUD completo + soft delete (marcar `eliminado=true`, nunca borrar físicamente).

05

Controller + AdviceController

Endpoints REST completos y manejo centralizado de errores con `@RestControllerAdvice`.

06

Probar con Postman y Swagger

2 autores, 5 libros, update de stock, búsqueda por autor, soft delete.

CAPA REPOSITORY

Repository — JpaRepository + queries derivadas

```
// AutorRepository.java
public interface AutorRepository
        extends JpaRepository<Autor, Long> {

    List<Autor> findByEliminadoFalse();
}

// LibroRepository.java
public interface LibroRepository
        extends JpaRepository<Libro, Long> {

    List<Libro> findByEliminadoFalse();

    List<Libro> findByAutor_IdAndEliminadoFalse(
            Long autorId);
}
```

¿Por qué queries derivadas en el Repository?

- Spring las traduce a SQL automáticamente — no escribís JPQL.
- El nombre de la query **documenta** su intención.
- El Service no sabe cómo se filtra — solo pide los datos.
- La lógica de acceso a datos queda encapsulada en su capa.

CAPA DTOS

Patrón Create / Dto / Edit — cada record trae su mapeo

LibroCreate

```
public record LibroCreate(
@NotBlank String titulo,
@NotNull @Positive Integer anio,
@NotNull @Positive Double precio, @NotNull
@PositiveOrZero Integer stock,
@NotNull EstadoLibro estado,
@NotNull Long autorId
) {
}
```

LibroDto

```
public record LibroDto(
        Long id,
        String titulo,
        Integer anio,
        Double precio,
        Integer stock,
        EstadoLibro estado,
        String autorNombreCompleto
) {

}
```

LibroEdit

```
public record LibroEdit(
  String titulo,
  Integer anio,
  Double precio,
  Integer stock,
  EstadoLibro estado
) {
}
```

CAPA SERVICE

Service — Soft Delete y operaciones CRUD

Implementación de soft delete

```
@Service
public class LibroServiceImpl implements LibroService {

    @Override
    public void delete(Long id) {
        Libro libro = libroRepo
            .findById(id)
            .orElseThrow(() ->
                new ResourceNotFoundException(
                    "Libro no encontrado: " + id));
        libro.setEliminado(true); // soft delete
        libroRepo.save(libro);
    }
}
```

Reglas del Service

Solo DTOs

Nunca devuelve entidades — siempre `LibroDto`, `AutorDto`, etc.

Sin HTTP

No importa `HttpServletRequest` ni conoce status codes.

Soft delete

`eliminado=true` y `save()`. Jamás `deleteById()`.

CAPA CONTROLLER + ADVICECONTROLLER

Controller REST y manejo centralizado de errores

LibroController

```
@RestController
@RequestMapping("/api/libros")
public class LibroController {

 @PostMapping
 public ResponseEntity<LibroDto> create(
    @RequestBody @Valid LibroCreate dto) {
  return ResponseEntity.status(201)
     .body(service.create(dto));
 }
 @DeleteMapping("/{id}")
 public ResponseEntity<Void> delete(
    @PathVariable Long id) {
  service.delete(id);
  return ResponseEntity.noContent().build();
 }
 // ... getById, update, getAll
}
```

AdviceController

```
@RestControllerAdvice
public class AdviceController{
 @ExceptionHandler(
    EntityNotFoundException.class)
 @ResponseStatus(HttpStatus.NOT_FOUND)
 public Map<String, String> handleNotFound(
    EntityNotFoundException ex) {
  return Map.of("error", ex.getMessage());
 }
 @ExceptionHandler(
    MethodArgumentNotValidException.class)
 @ResponseStatus(HttpStatus.BAD_REQUEST)
 public Map<String, Object> handleValidation(
    MethodArgumentNotValidException ex) {
  // extrae errores campo a campo
 }
 // ... más handlers
}
```

CONCEPTO CLAVE

¿Por qué el Service nunca expone la entidad directamente?

❌ Sin DTOs — el Service expone la entidad

```
// La entidad llega hasta el Controller
public Libro getById(Long id) {
    return libroRepo.findById(id)
        .orElseThrow(...);
}
// Problemas:
// - Jackson serializa TODOS los campos
// - Se expone "eliminado", "createdAt", etc.
// - Cambiar la entidad rompe la API
// - Las relaciones lazy explotan al serializar
```

✅ Con DTOs — el Service expone solo lo necesario

```
// El DTO controla exactamente qué se devuelve
public LibroDto getById(Long id) {
    Libro libro = libroRepo.findById(id)
        .orElseThrow(...);
    return LibroDto.fromEntity(libro);
}
// Beneficios:
// - Solo los campos que el cliente necesita
// - La entidad puede cambiar sin romper la API
// - Sin problemas de serialización lazy
// - El contrato de la API es explícito
```

Encapsulamiento

La entidad es un detalle de implementación — el cliente no necesita saber que existe.

Estabilidad del contrato

Podés refactorizar la entidad sin romper a quienes consumen la API.

Seguridad

Evitás exponer campos internos como `eliminado`, contraseñas hasheadas, etc.

CONCEPTO CLAVE

@Valid solo en el Create — y por qué centralizar los errores

¿Por qué @Valid solo en el Create?

En el Edit los campos son opcionales — `@Valid` con `@NotBlank` rechazaría updates parciales válidos. Solo el Create exige todos los campos.

```
// Create: todos los campos requeridos
@PostMapping
public ResponseEntity<LibroDto> create(
    @RequestBody @Valid LibroCreate dto) { ... }

// Edit: campos opcionales, sin @Valid
@PutMapping("/{id}")
public LibroDto update(@PathVariable Long id,
    @RequestBody LibroEdit dto) { ... }
```

¿Por qué centralizar los errores?

Sin `@RestControllerAdvice` cada endpoint necesita su propio try/catch. Con el Advice el manejo es uniforme y el Controller queda limpio.

```
// Sin Advice: repeticion en cada endpoint
@GetMapping("/{id}")
public LibroDto getById(@PathVariable Long id) {
  try {
    return service.getById(id);
  } catch (EntityNotFoundException e) {
    throw new ResponseStatusException(
        NOT_FOUND, e.getMessage());
  }
}
// Con Advice: Controller limpio
@GetMapping("/{id}")
public LibroDto getById(@PathVariable Long id) {
  return service.getById(id);
}
```

SWAGGER / OPENAPI

Documentación automática de la API con Swagger

1. Agregar dependencia

```
//build.gradle
implementation 'org.springdoc:springdoc-openapi-starter-webmvc-ui:2.5.0'
```

2. Acceder a la UI

```
http://localhost:8080/swagger-ui.html
```

¡Sin configuración adicional! Spring detecta los controllers automáticamente.

¿Qué aporta Swagger a la API?

Exploración interactiva

El frontend puede probar endpoints directamente desde el navegador sin Postman.

Contrato explícito

Documenta qué recibe cada endpoint y qué devuelve — en tiempo real.

Validación visual

Podés ver los modelos de request/response y los códigos de error posibles.

PRUEBA CON POSTMAN

Secuencia de pruebas con Postman

1

POST /api/autores × 2

Crear 2 autores. Guardá los `id` de la respuesta.

2

POST /api/libros × 5

Crear 5 libros, repartidos entre los 2 autores usando sus `autorId`.

3

PUT /api/libros/{id}

Actualizar el `stock` de un libro. Solo ese campo — el Edit es parcial.

4

GET /api/libros/{id}

Buscar un libro por ID. Verificar que devuelve el stock actualizado.

5

GET /api/autores/{id}/libros

Traer todos los libros de un autor — usa la query derivada findByAutor_IdAndEliminadoFalse.

6

DELETE /api/libros/{id}

Soft delete. Verificar con GET que ya no aparece. Confirmar en la BD que `eliminado=true`.

💡 Por último, abrí `swagger-ui.html` y mostrá que todos los endpoints están documentados automáticamente.